jQuery and Interactive Web Features

Assignment 6 - Full Website Report

Atlas Fitness Club

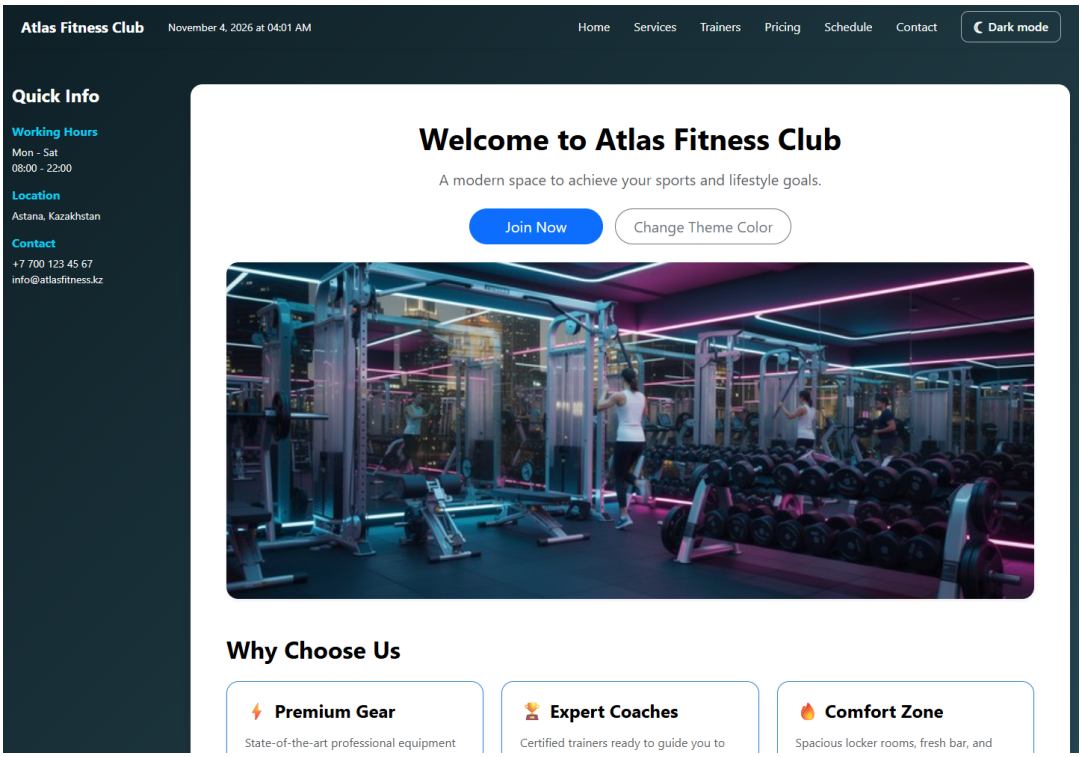
Team: Atlas Fitness Club

Members: Nursultan Akhat, Abdugani Nagashbay, Tanat Azamat

Group: SE-2506

Scope: Home, Services, Trainers, Pricing, Schedule and Contact

Date: 4 November 2026



The current Home page. The report covers additions across the complete team website.

Project website: https://nursultanakhat.github.io/fitness_club/

Repository: https://github.com/nursultanakhat/fitness_club

The report documents the current local project, with separate verification and deployment notes. Existing issues are recorded in Section 13.

1. Objective and project overview

The objective was to continue the Atlas Fitness Club project with jQuery features for searching, engagement and user feedback. This report covers the work across all six pages of the website, including the team additions already present in the supplied project and the later Schedule and Contact changes.

The Assignment 5 ZIP was used as the earlier baseline. The current project contains updated versions of all six HTML pages, the shared JavaScript and the stylesheet. Code figures use the current source line numbers; webpage figures are actual captures of the local site.

Additions by page

Page	Assignment 6 additions
1. Home	Scroll progress bar (Task 4), jQuery loading and saved light/dark theme.
2. Services	Autocomplete (Task 2), highlighting (Task 3), scroll progress (Task 4), submit spinner (Task 6), toast feedback (Task 7) and theme support.
3. Trainers	Animated counters (Task 5), lazy-loaded images (Task 9), theme support and improved booking-dialog interaction.
4. Pricing	Copy membership quote (Task 8), jQuery calculator validation and theme support.
5. Schedule	Real-time timetable search (Task 1), matching text highlights (Task 3), combined filters and reset.
6. Contact	Submit spinner (Task 6), phone/email copy buttons (Task 8), validation and two-step form recovery.

Task 0 provides the jQuery setup. Tasks 1-9 are represented across the site; some tasks appear on more than one page. Services search suggests and highlights text but does not hide cards. The live filter is demonstrated on Schedule.

Evidence and limits

Screenshots show recorded interface states. Browser checks additionally verify clipboard contents, animation values, deferred image requests and timed form states. Known Services errors are reported rather than treating the complete website as error-free.

2. Task 0 - jQuery setup

All six pages load jQuery 4.0.0 before the shared script. Schedule and Contact include the CDN script in the head and a bundled fallback; the other four pages retain their existing footer loading order.

Step 1: Include jQuery before code that uses the $ function.

Step 2: Initialize interactions inside $(document).ready() and print "jQuery is ready!" in the console.

Step 3: On Schedule and Contact, check window.jQuery and load the local library if the CDN fails.

Implementation: schedule.html (also used in contact.html)

```
schedule.html | source excerpt | lines 12-20
  12 <!-- Task 0: CDN first, then the bundled copy if the CDN is unavailable. -->
  13 <script src="https://code.jquery.com/jquery-4.0.0.min.js"
  14         integrity="sha384-fgGyf7Mo7DURSOMnOy7ed+dkq5Job205Gnzu6QIg0BOHKaqt4D76Dt8VlDCzcMHV"
  15         crossorigin="anonymous"></script>
  16 <script>
  17     if (!window.jQuery) {
  18         document.write('<script src="js/vendor/jquery-4.0.0.min.js"><\/script>');
  19     }
  20 </script>
```

Implementation: script.js and js/schedule-contact.js

```
script.js | source excerpt | lines 411-412
 411 $(document).ready(function() {
 412     console.log("jQuery is ready!");

js/schedule-contact.js | source excerpt | lines 1-6
   1 /* Assignment 6: Tasks 1, 3, 6 and 8. Loaded only by pages 5 and 6. */
   2 (function ($) {
   3     'use strict';
   4
   5     $(document).ready(function () {
   6         if (!$('body').hasClass('assignment6-page')) return;
```

Dependency verification

The pages loaded jQuery 4.0.0. Home and Services retain Bootstrap 5.3.0; the other four pages use 5.3.3. Blocking the jQuery CDN still allowed Schedule and Contact to initialize from js/vendor/jquery-4.0.0.min.js. The other pages and Bootstrap continue to rely on CDN connections.

The assignment requests the library in the head. The current project meets that placement on Schedule and Contact; Home, Services, Trainers and Pricing still place it near the end of the body.

3. Task 1 - Real-time search and live filter

The Schedule page filters the weekly timetable while the visitor types. The text query and the existing training-type dropdown apply together.

Step 1: Trim the search text and compare class names without case sensitivity.

Step 2: Check each class against both controls and preserve the original weekday cells.

Step 3: Update the matching count. Reset clears both controls and restores all 36 sessions.

Implementation: js/schedule-contact.js

```
js/schedule-contact.js | source excerpt | lines 38-64
function filterSchedule() {
    const keyword = $search.val().trim().toLowerCase();
    const selectedType = $type.val().toLowerCase();
    let count = 0;
    $cells.each(function () {
        const $cell = $(this);
        const text = $cell.data('className');
        const lowerText = text.toLowerCase();
        const matches = (selectedType === 'all' || lowerText === selectedType)
            && lowerText.includes(keyword);
        $cell.toggleClass('schedule-visible', matches)
            .toggleClass('schedule-hidden', !matches);
        if (matches) {
            count += 1;
            highlight($cell, text, keyword);
        } else {
            // Keep each weekday column in place, including empty results.
            $cell.empty();
        }
    });
    const message = !keyword && selectedType === 'all'
        ? 'Showing all ' + count + ' training classes.'
        : count === 0
            ? 'No classes match your search. Try another name or reset the filters.'
            : 'Found ' + count + ' matching ' + (count === 1 ? 'class.' : 'classes.');
    $('#scheduleFilterResult').text(message);
}
```

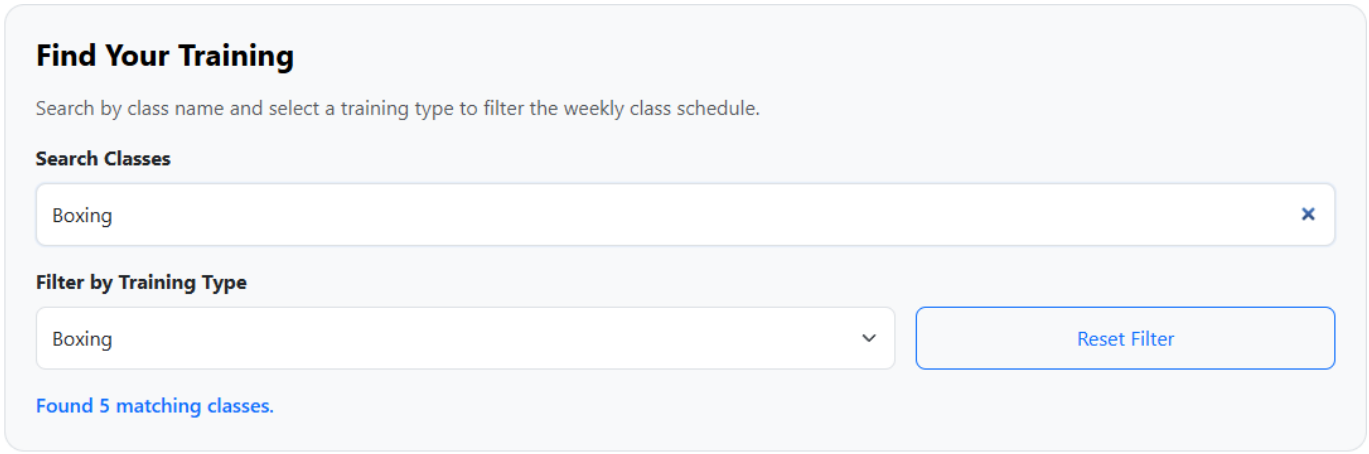
Search, type selection and reset events

```
js/schedule-contact.js | source excerpt | lines 66-74
$search.on('input.assignment6', filterSchedule);
$type.on('change.assignment6', filterSchedule);
$('#resetScheduleFilter').on('click.assignment6', function () {
    $search.val('');
    $type.val('all');
    filterSchedule();
    $search.trigger('focus');
});
filterSchedule();
```

3.1 Schedule search results

The result below combines a Boxing query with Boxing selected in the dropdown. Five sessions remain visible at their original time and weekday positions.

Webpage result



Time	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday
08:00						
10:00		Boxing		Boxing		
12:00						
15:00						Boxing
18:00	Boxing			Boxing		
20:00						

Actual Schedule state: the two filters return five Boxing classes.

How to reproduce the result

Step 1: Open Schedule and type Boxing in Search Classes.

Step 2: Select Boxing in the training dropdown; the matching count remains five.

Step 3: Type Yoga while keeping Boxing selected. The no-results message appears.

Step 4: Click Reset Filter to restore all 36 classes.

The input event handles typed and pasted text. The query " yOgA " returns eight Yoga sessions. This is distinct from the Services suggestions, which leave all three service cards visible.

4. Task 2 - Autocomplete suggestions

The Services search field displays matching suggestions from a predefined list of club services and workouts.

Step 1: Read the keyword on keyup and clear the previous suggestion list.

Step 2: Filter the available labels without case sensitivity and append matching dropdown items.

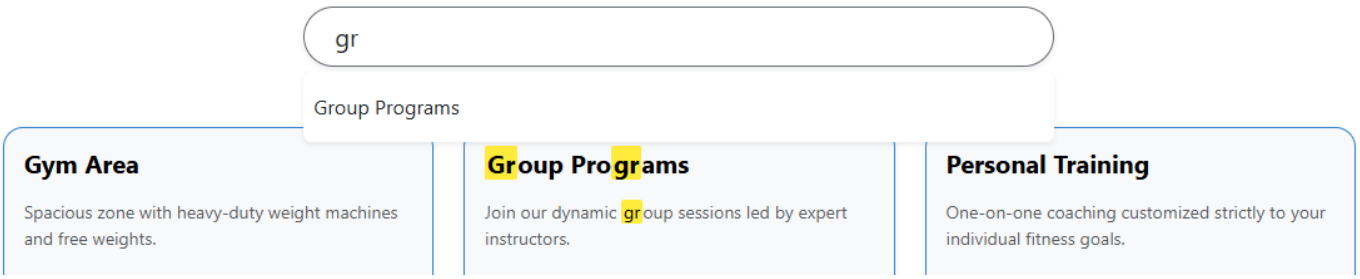
Step 3: When a suggestion is clicked, copy it into the input, trigger the search logic and close the dropdown.

Implementation: script.js

```
script.js | source excerpt | lines 426-442
  426 $('#searchInput').on('keyup', function() {
  427     let keyword = $(this).val().trim();
  428     let $suggestionsBox = $('#suggestions');
  429
  430     $suggestionsBox.empty();
  431
  432     let suggestionsList = ["Gym Area", "Group Programs", "Personal Training", "Yoga and
        Stretching", "CrossFit", "Pilates"];
  433
  434     if (keyword.length > 0) {
  435         let matches = suggestionsList.filter(item =>
        item.toLowerCase().includes(keyword.toLowerCase()));
  436         matches.forEach(match => {
  437             $suggestionsBox.append(`<div class="suggestion-item dropdown-item p-2"
        style="cursor: pointer;">${match}</div>`);
  438         });
  439         $suggestionsBox.show();
  440     } else {
  441         $suggestionsBox.hide();
  442     }
```

```
script.js | source excerpt | lines 463-466
  463 $(document).on('click', '.suggestion-item', function() {
  464     $('#searchInput').val($(this).text()).trigger('keyup');
  465     $('#suggestions').hide();
  466 });
```

Webpage result



The Services input shows a matching suggestion before selection.

The dropdown was checked with a normal search term and by clicking a suggestion. It provides suggestions; it does not filter the service cards themselves.

5. Task 3 - Highlighting on Services

Services highlights matches inside the headings and paragraphs of the service cards. The code removes old mark elements before applying a new search.

Step 1: Replace existing mark elements with their text.

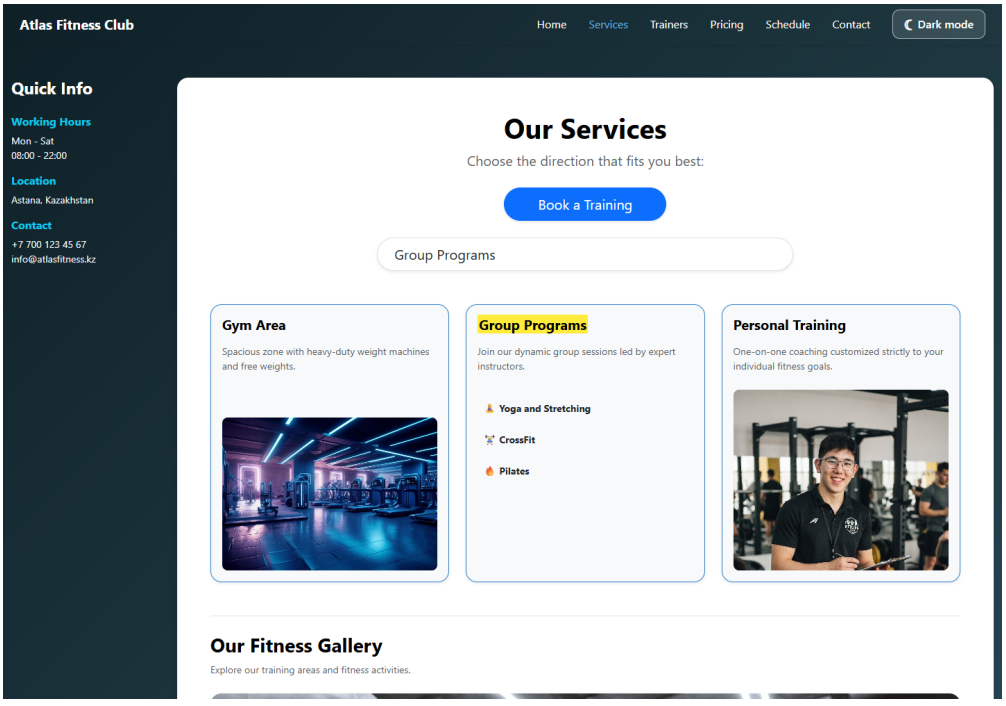
Step 2: Build a case-insensitive regular expression for the keyword.

Step 3: Wrap matching substrings with a styled mark element and update the card content.

Implementation: script.js

```
script.js | source excerpt | lines 445-447, 450-460
  445     $('.service-card mark').each(function() {
  446         $(this).replaceWith($(this).text());
  447     });
      ... source lines 448-449 omitted ...
  450     if (keyword.length > 0) {
  451         let regex = new RegExp('(' + keyword + ')', 'gi');
  452         $('.service-card h3, .service-card p').each(function() {
  453             let html = $(this).html();
  454             if (!html.includes('mark')) {
  455                 let newHtml = html.replace(regex, '<mark style="background-color: #ffeb3b;
          padding: 0 2px; border-radius: 2px;">$1</mark>');
  456                 $(this).html(newHtml);
  457             }
  458         });
  459     }
  460 });
```

Webpage result



A normal Services keyword produces visible highlighted text in the matching card content.

Current limitation: the keyword is not escaped before constructing the regular expression. Entering [reproduced an error. Schedule uses a separate literal-text implementation, shown next.

5.1 Highlighting on Schedule

Schedule highlights every matching occurrence within the visible timetable class names. It keeps original names in memory so filters can be cleared without losing content.

Step 1: Locate each occurrence with indexOf().

Step 2: Create a mark element for the matching substring and append text nodes around it.

Step 3: Clear the input to restore the plain original text.

Implementation: js/schedule-contact.js

```
js/schedule-contact.js | source excerpt | lines 18-36
function highlight($cell, text, keyword) {
    $cell.empty();
    if (!keyword) {
        $cell.text(text);
        return;
    }
    const lowerText = text.toLowerCase();
    let position = 0;
    let match = lowerText.indexOf(keyword);
    while (match !== -1) {
        $cell.append(document.createTextNode(text.slice(position, match)));
        // User input is never interpreted as HTML or a regular expression.
        $('<mark>', { class: 'schedule-search-match' })
            .text(text.slice(match, match + keyword.length)).appendTo($cell);
        position = match + keyword.length;
        match = lowerText.indexOf(keyword, position);
    }
    $cell.append(document.createTextNode(text.slice(position)));
}
```

Webpage result

Time	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday
08:00	Yoga	CrossFit	Yoga	CrossFit	Yoga	
10:00		Boxing		Boxing		CrossFit
12:00		Yoga		Yoga		
15:00	CrossFit		CrossFit		CrossFit	Boxing
18:00	Boxing		Yoga	Boxing	Yoga	
20:00		CrossFit			CrossFit	Yoga

The partial query o highlights occurrences in Yoga, CrossFit and Boxing.

This implementation treats user input as literal text, not HTML or regular-expression syntax. The browser checks included [, (, .* and HTML-looking input without executing markup or throwing an error.

6. Task 4 - Scroll progress bar

Home and Services contain a fixed progress bar at the top of the viewport. Its blue-to-cyan gradient fills as the visitor scrolls down the page.

Step 1: Read the current vertical scroll position.

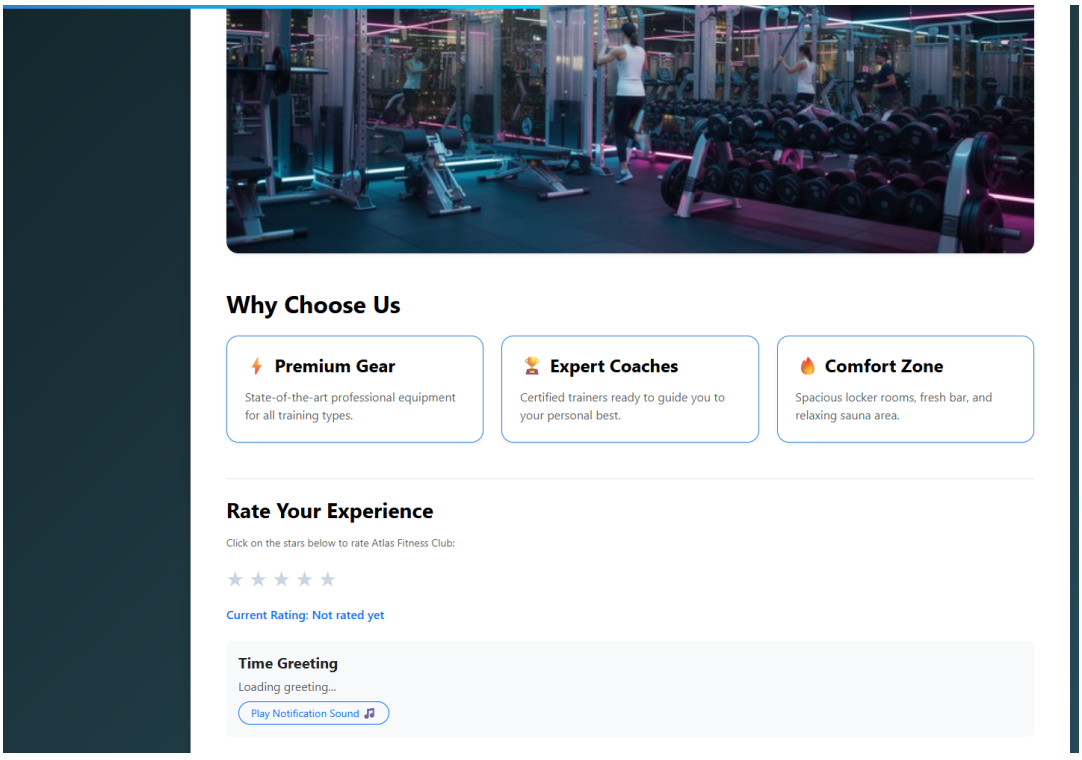
Step 2: Divide it by the scrollable document height and convert the result to a percentage.

Step 3: Use jQuery .css() to update the progress bar width on each scroll event.

Implementation: script.js

```
script.js | source excerpt | lines 469-474
  469 $(window).on('scroll', function() {
  470     let sTop = $(window).scrollTop();
  471     let docHeight = $(document).height() - $(window).height();
  472     let scrollPercent = (sTop / docHeight) * 100;
  473     $('#scroll-progress-bar').css('width', scrollPercent + '%');
  474 });
```

Webpage result: Home



The fixed bar reflects the recorded scroll position on the Home page.

Detail: the same progress feature on Services



Home was checked at the top, an intermediate position and the bottom; Services was checked at the bottom. The observed width followed the scroll position. The shared handler updates pages containing the progress-bar element.

7. Task 5 - Animated number counters

The Trainers page adds three statistics: Professional Trainers, Training Specialties and Club Opening Hours per Week.

Step 1: Start a counter when its element enters the viewport and record that it has started.

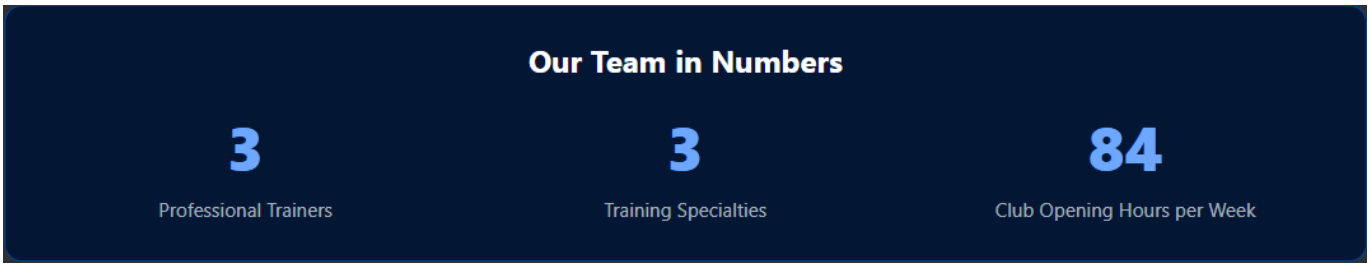
Step 2: Animate a numeric value from zero to data-target over 1,800 ms.

Step 3: Write intermediate whole numbers and then the exact final value. Reduced-motion preference skips the animation.

Implementation: script.js

```
script.js | source excerpt | lines 636-654
  636 const target = Number($counter.attr("data-target"));
  637
  638 $counter.text("0");
  639
  640 $({ value: 0 }).animate(
  641     { value: target },
  642     {
  643         duration: prefersReducedMotion ? 0 : 1800,
  644         easing: "swing",
  645
  646         step: function () {
  647             $counter.text(
  648                 Math.floor(this.value).toLocaleString("en-US")
  649             );
  650         },
  651
  652         complete: function () {
  653             $counter.text(target.toLocaleString("en-US"));
  654         }
```

Webpage result



The three counters finish at 3 trainers, 3 specialties and 84 opening hours per week.

The browser recorded intermediate values before the final totals. Scrolling away and back did not restart the counters. These elements are on Trainers; Pricing does not contain counter markup.

8. Task 6 - Contact loading state

The Contact registration form displays a spinner and waiting text after valid details are submitted.

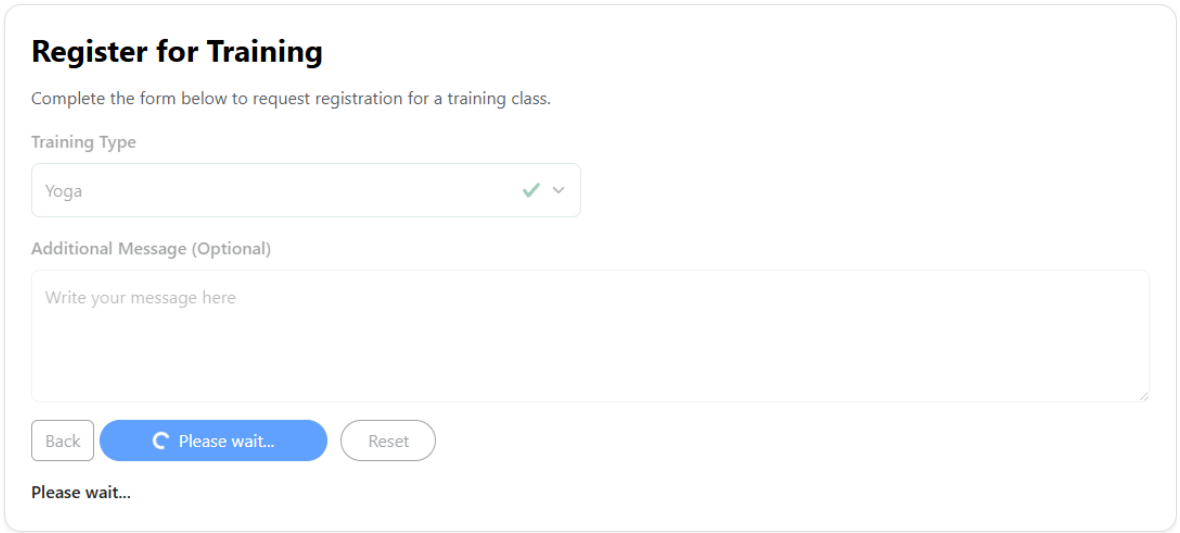
Step 1: Use one submit handler, validate the fields and block repeated submits with a busy flag.

Step 2: Disable the controls, show the spinner and restore the form after a 1,500 ms simulated delay.

Implementation: js/schedule-contact.js

```
js/schedule-contact.js | source excerpt | lines 148-150, 164-180
  148 $form.on('submit.assignment6', function (event) {
  149     event.preventDefault();
  150     if (busy) return;
      ... source lines 151-163 omitted ...
  164     busy = true;
  165     controlStates = $form.find('input, select, textarea, button').toArray().map(function
          (element) {
  166         const state = { element: element, disabled: element.disabled };
  167         element.disabled = true;
  168         return state;
  169     });
  170     $form.attr('aria-busy', 'true');
  171     $submit.html('<span class="spinner-border spinner-border-sm me-2"
          aria-hidden="true"></span>Please wait...');
  172     $message.attr('class', 'mt-3').text('Please wait...');
  173
  174     // The assignment asks for a simulated server delay; no data is sent.
  175     pendingTimer = setTimeout(function () {
  176         $form[0].reset();
  177         $message.attr('class', 'alert alert-success mt-3')
  178             .text('Demo registration completed successfully. No registration has been sent.');
  179     }, 1500);
  180 });
```

Webpage result



The Contact button shows Please wait... while the form controls are disabled.

Next, Back and Reset remain available in the normal form flow. A reset cancels pending completion. This demonstration sends no registration request to a server.

8.1 Services loading state

The Services booking popup also uses a submit spinner. It validates the booking fields, disables the button and shows Please wait... before the delayed callback completes.

Step 1: Select #submit-btn with jQuery and set its disabled property.

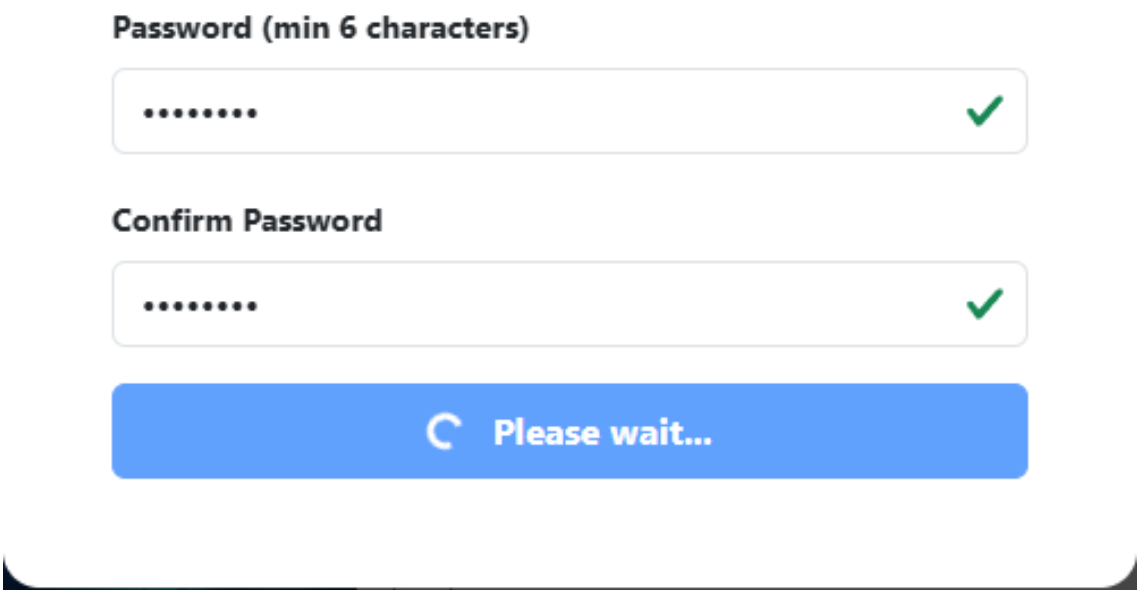
Step 2: Replace the button content with the Bootstrap spinner and waiting text.

Step 3: After the delay, restore Submit Booking and close the popup through the completion callback.

Implementation: script.js

```
script.js | source excerpt | lines 523-532, 545-548, 554-554
  523 if (isValid) {
  524     const $btn = $('#submit-btn');
  525
  526     if ($btn.length > 0) {
  527         $btn.prop('disabled', true);
  528         $btn.html(`
  529             <span class="spinner-border spinner-border-sm me-2" role="status"
      aria-hidden="true"></span>
  530             Please wait...
  531         `);
  532     }
      ... source lines 533-544 omitted ...
  545         if ($btn.length > 0) {
  546             $btn.prop('disabled', false);
  547             $btn.html('Submit Booking');
  548         }
      ... source lines 549-553 omitted ...
  554     }, 1500);
```

Webpage result



Detail of the actual Services popup during the waiting state.

The visible loading state works, but this form currently has two native submit handlers. One delayed callback throws showToast is not defined; the other still completes the visible reset and notification. This integration issue remains in the supplied code.

9. Task 7 - Temporary notification

Services adds a toast notification after the simulated booking validation finishes. The notification gives immediate feedback and disappears automatically.

Step 1: Write the message into #toast-message-text.

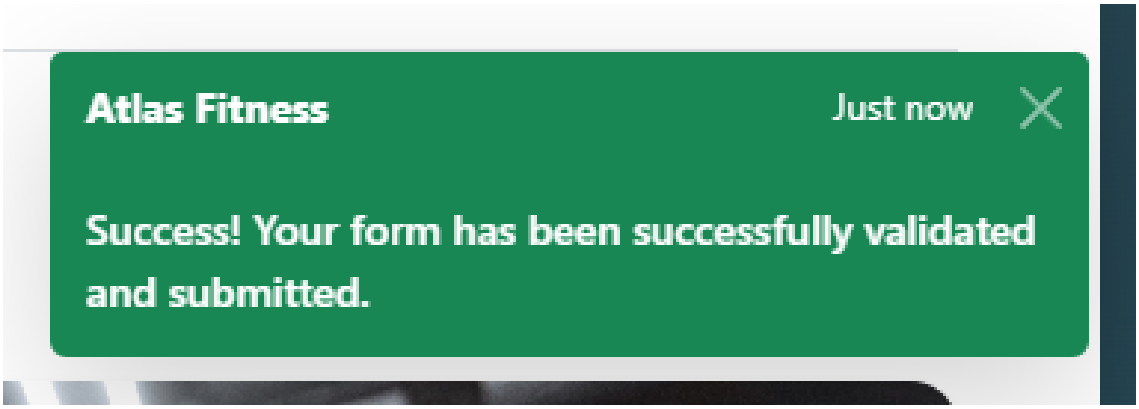
Step 2: Reveal the notification with fadeIn().

Step 3: Use setTimeout() and fadeOut() to hide it after 3.5 seconds; the close button can hide it earlier.

Implementation: script.js

```
script.js | source excerpt | lines 415-423, 534-538
  415 function showToast(message) {
  416     const $toast = $('#custom-toast');
  417     $('#toast-message-text').text(message);
  418     $toast.fadeIn();
  419
  420     setTimeout(function() {
  421         $toast.fadeOut();
  422     }, 3500);
  423 }
      ... source lines 424-533 omitted ...
  534             setTimeout(function() {
  535                 // Вызываем наше красивое уведомление (Task 7)
  536                 showToast('Success! Your form has been successfully validated and submitted.');
  537
  538                 bookingForm.reset();
```

Webpage result



The Services success notification shown by the working completion callback.

The test observed the toast appear and later become hidden. Its success wording refers to the local demonstration; the form has no backend submission. The duplicate-handler error described in Section 8.1 occurs independently of this visible notification.

10. Task 8 - Copy a membership quote

The Pricing page adds Copy quote beside the membership calculator. It copies the current plan, duration calculation and total price.

Step 1: Build the quote from the selected plan and the displayed calculation.

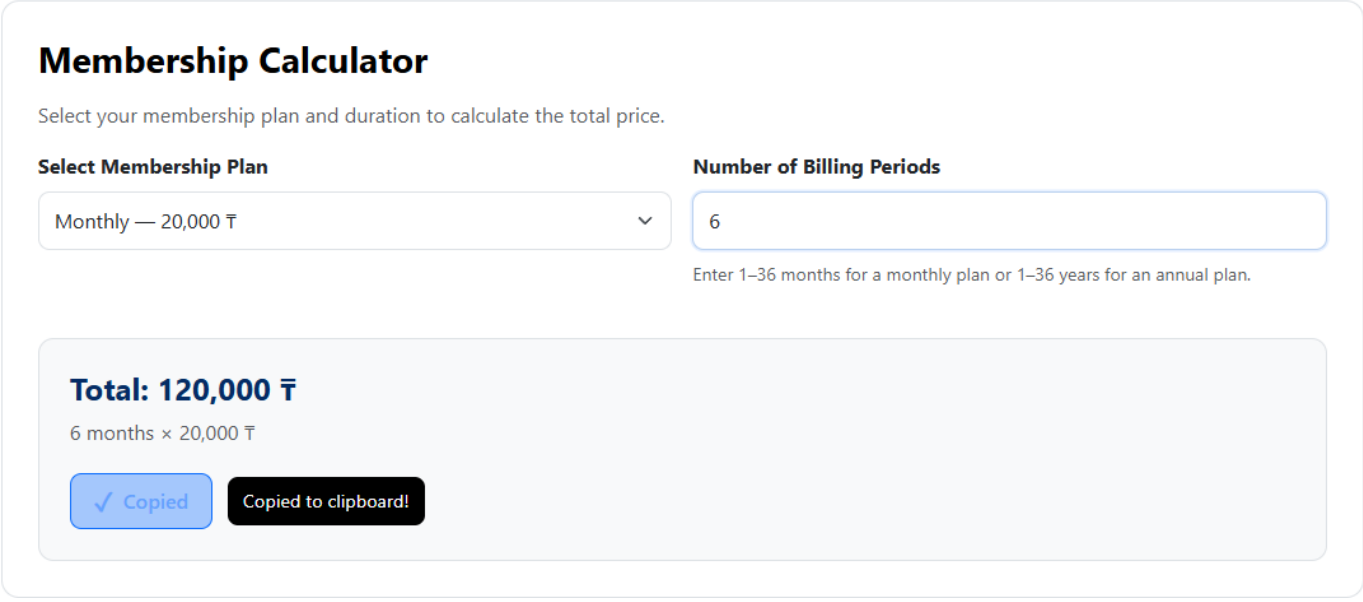
Step 2: Copy the text, then show a checkmark and Copied to clipboard! feedback.

Step 3: Restore the normal label after three seconds. Clear stale feedback when the quote changes.

Implementation: script.js

```
script.js | source excerpt | lines 826-845
  826     "Atlas Fitness Club",
  827     planName,
  828     $("#membershipExplanation").text(),
  829     $("#membershipTotal").text()
  830 ].join("\n");
  831
  832 copying = true;
  833 $copyButton.prop("disabled", true);
  834
  835 try {
  836     await copyText(quote);
  837
  838     if (copiedVersion === quoteVersion) {
  839         $copyButton.find(".copy-icon").text("✓");
  840         $copyButton.find(".copy-label").text("Copied");
  841
  842         $copyFeedback
  843             .text("Copied to clipboard!")
  844             .addClass("is-visible");
  845     }
```

Webpage result



Six monthly periods produce 120,000 KZT; the same amount was read back from the clipboard.

The calculator validates a whole-number duration from 1 to 36. Copy is disabled for invalid input. The copied text was checked, not only the visual feedback.

10.1 Copy contact details

Contact adds Copy buttons next to the club phone number and email address.

Step 1: Read the target text from the element referenced by data-copy-target.

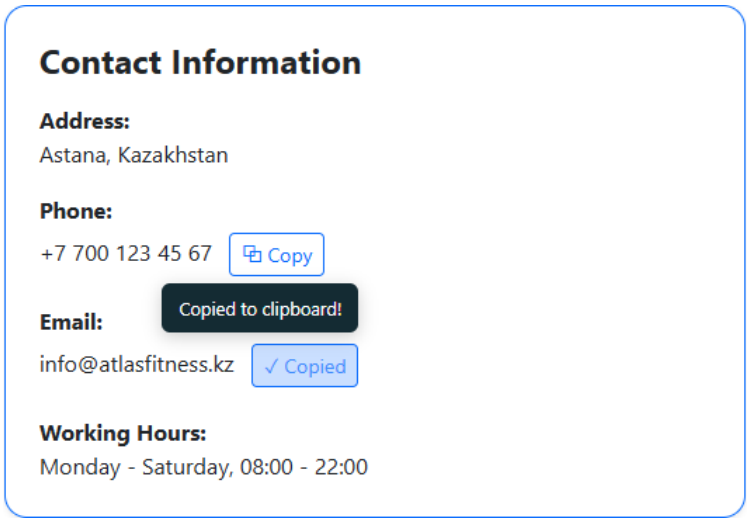
Step 2: Use navigator.clipboard.writeText() with a temporary-textarea fallback.

Step 3: Show a checkmark and success tooltip only after copying succeeds; restore the label after 2.5 seconds.

Implementation: js/schedule-contact.js

```
js/schedule-contact.js | source excerpt | lines 206-225
  206 $('.a6-copy-button').on('click.assignment6', async function () {
  207     const $button = $(this);
  208     if ($button.data('copying')) return;
  209     const $tooltip = $('#' + $button.attr('aria-describedby'));
  210     const text = $('#' + $button.attr('data-copy-target')).text().trim();
  211     clearTimeout($button.data('feedbackTimer'));
  212     $button.data('copying', true);
  213     try {
  214         if (window.isSecureContext && navigator.clipboard && navigator.clipboard.writeText) {
  215             try {
  216                 await navigator.clipboard.writeText(text);
  217             } catch (error) {
  218                 legacyCopy(text);
  219             }
  220         } else {
  221             legacyCopy(text);
  222         }
  223         $button.find('.a6-copy-icon').text('✓');
  224         $button.find('.a6-copy-label').text('Copied');
  225         $tooltip.text('Copied to clipboard!');
```

Webpage result



The email button changes to Copied and displays the success tooltip.

The phone and email clipboard contents were verified. If copying fails, the interface asks the visitor to copy the text manually instead of displaying false success.

11. Task 9 - Image lazy loading

Trainers contains seven lazy-image elements, including the four-image gallery. Their real image paths are stored in data-src while the initial src is a lightweight placeholder.

Step 1: Compare each image position with the visible viewport.

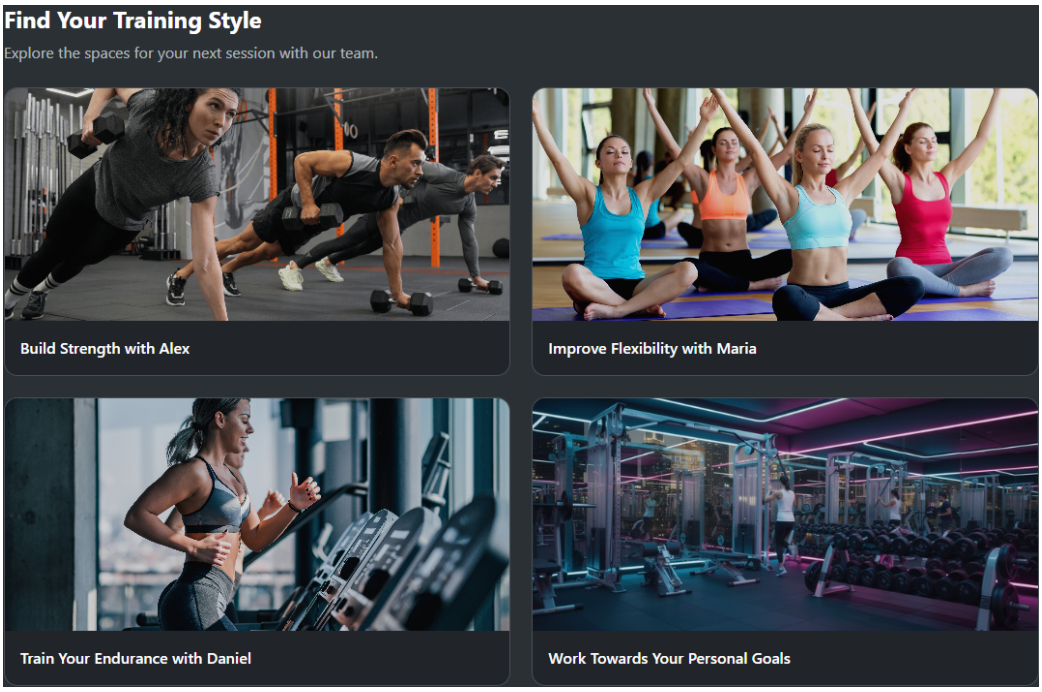
Step 2: When the image enters view, move data-src into src and remove data-src.

Step 3: On load, add is-loaded; on failure, expose an Image unavailable alternative text.

Implementation: script.js

```
script.js | source excerpt | lines 659-670, 680-683
  659 $(".fitness-lazy[data-src]").each(function () {
  660     const $image = $(this);
  661     const imageTop = $image.offset().top;
  662     const imageBottom = imageTop + $image.outerHeight();
  663
  664     if (
  665         imageTop < viewportBottom &&
  666         imageBottom > viewportTop
  667     ) {
  668         const source = $image.attr("data-src");
  669
  670         $image.one("load", function () {
      ... source lines 671-679 omitted ...
  680         $image
  681             .removeAttr("data-src")
  682             .attr("src", source);
  683     }
```

Webpage result



The Trainers gallery after scrolling it into view and loading the real images.

For strength.jpg, the test recorded a placeholder and no image request before scrolling. After scrolling, data-src was removed, the real request occurred and the image had a non-zero natural width.

12. Theme and supporting improvements

All six pages now share a light/dark theme control. The preference is stored under atlas-theme and applied through Bootstrap's data-bs-theme attribute.

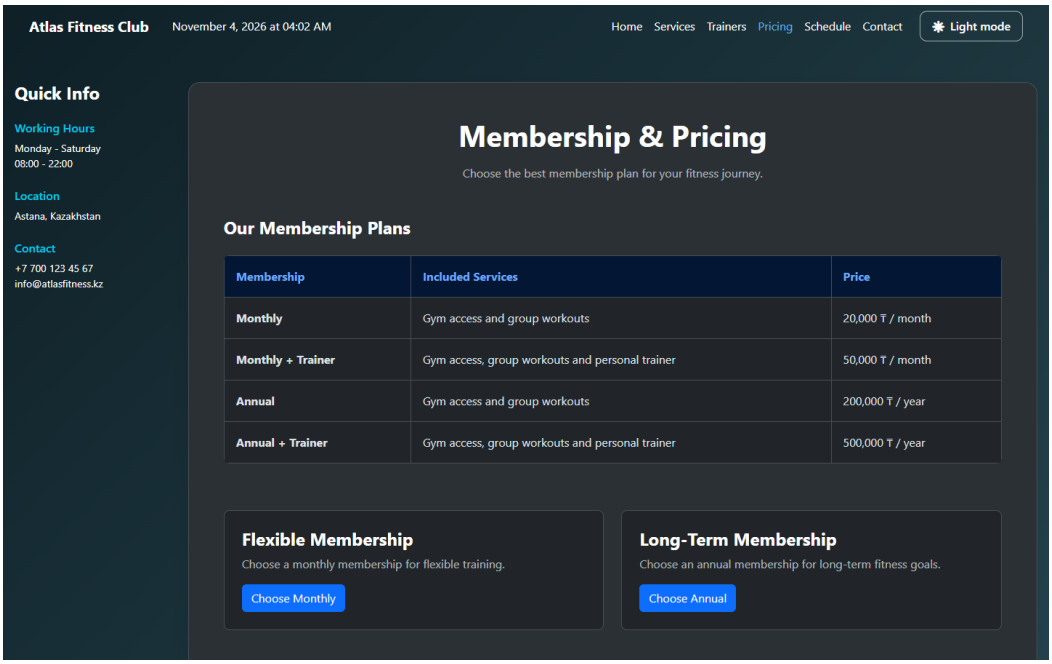
Step 1: Apply the selected theme and update the button icon, label and accessibility attributes.

Step 2: Save the preference in localStorage and restore it on later page visits.

Implementation: script.js

```
script.js | source excerpt | lines 569-584
  569 function applyTheme(theme) {
  570     $root.attr("data-bs-theme", theme);
  571
  572     const isDark = theme === "dark";
  573
  574     $("#theme-icon").text(isDark ? "☀" : "☾");
  575     $("#theme-label").text(isDark ? "Light mode" : "Dark mode");
  576
  577     $themeButton.attr({
  578         "aria-label": isDark
  579             ? "Switch to light theme"
  580             : "Switch to dark theme",
  581         "aria-pressed": String(isDark)
  582     });
  583 }
  584
```

Webpage result: Pricing in dark mode



The current shared theme keeps the existing page structure while changing its colour scheme.

The Pricing calculator was integrated with the new copy-quote state. The Trainers dialog now supports initial focus, Escape, Tab/Shift+Tab focus containment and focus return after closing. Its confirmation explicitly identifies the booking as a demo. These are supporting updates, not extra numbered assignment tasks.

13. Testing and current limitations

The complete-site review combines source comparison, dependency and syntax checks, and browser checks on each page group. Results are reported separately because Services has reproduced runtime defects.

Page group	Recorded verification
Home / Services	28 audit observations, including autocomplete selection, normal-word highlighting, progress widths, spinner and toast states, themes and the two defect reproductions below.
Trainers / Pricing	32 of 32 checks passed with no uncaught errors: counter transitions, lazy-image requests, real quote copying, invalid duration, themes and dialog keyboard behaviour.
Schedule / Contact	43 checks passed with no uncaught errors: filter combinations, text safety, form state, one contact submit handler, real contact copying, mobile layout and jQuery fallback.

Issues observed on Services

1. Entering [in the search field constructs an invalid regular expression and throws an error. Normal text highlighting works, but the keyword should be escaped before constructing the expression.

2. The booking form registers two native submit handlers. One callback references showToast outside its scope and throws showToast is not defined. The second callback still shows the toast, resets the form and closes the popup. Submission should be consolidated into one handler.

Scope of the verification

JavaScript and inline-script syntax checks passed. No missing local references or duplicate HTML IDs were found. Screenshots and browser checks document the tested paths; they do not establish that every inherited Assignment 5 feature is error-free. The site code was not changed during preparation of this full-site report.

A dependency review also found duplicate Bootstrap bundle imports on Trainers and Pricing. These should be consolidated to one import per page.

The forms simulate registration and booking. There is no backend submission, and clipboard access can depend on browser permissions. Bootstrap and the four original pages' jQuery imports still use CDN connections.

14. Structure, deployment and reflection

Project organization

Location	Purpose
Six HTML pages	Existing content plus the new search, progress, counter, clipboard and theme elements.
script.js	Shared jQuery features for Home, Services, Trainers and Pricing; theme handling across the site.
js/schedule-contact.js	Schedule filtering/highlighting and Contact form/copy functionality.
css/style.css and css/schedule-contact.css	Team site styling plus scoped additions for Schedule and Contact.
images/ and js/vendor/	Existing photographs and the local jQuery fallback used on Schedule/Contact.

Deployment

Project website: https://nursultanakhat.github.io/fitness_club/

Repository: https://github.com/nursultanakhat/fitness_club

The public Schedule and Contact pages returned HTTP 200 during verification on 4 November 2026, but did not contain the latest search input, copy controls or js/schedule-contact.js. The report therefore documents the full local project. The current ZIP must be published before those latest additions can be described as live.

Reflection and conclusion

The project applies jQuery to all three assignment areas: search, engagement and application functionality. The shared theme and new interactions continue the existing fitness-club design. Page-specific checks help prevent unrelated code from acting on the wrong page.

The review also shows why integration matters. The Schedule and Contact initializers were moved out of nested DOMContentLoaded listeners, while the Services duplicate submission remains an issue to resolve. Further work should fix the two Services errors, align the remaining jQuery imports with the requested head placement, deploy the current version and test additional browsers.

References

Assignment 6 jQuery(1).pdf: Tasks 0-9. Assignment 5 ZIP and report: earlier baseline and layout model. Current team website source and recorded local browser results: implementation evidence.

jQuery API: .ready() | jQuery API: .on()